High-Level Design (HLD)

MedInfo Assistant - Medical Research & Report Analysis Multi-Agent GenAI Platform

Generated 2026-10-15 from 02-HLD.md - information only, not medical advice.

Contents

0. Document control
1. Purpose and scope
2. System context
3. Architecture overview and principles
4. Logical components (container view)
5. End-to-end flows
6. Data architecture
7. Integration architecture
8. Security architecture
9. Deployment architecture
10. Non-functional design
11. Observability design
12. Technology stack and rationale
13. Constraints, assumptions and risks
14. Design decisions
15. Traceability (PRD goals -> HLD components)

0. Document control

Field	Value
Document	High-Level Design
Version	2.0 (production-grade)
Status	Draft for approval
Date	2026-10-14
Inputs	PRD v2.0 (source of truth)
Outputs	Architecture, LLD, API, Data model
Reviewers	Engineering Lead, Information Security, Clinical Safety Officer

Status labels follow the PRD: **Implemented** (built + offline tests), **Verified live** (real services, 2026-10-14), **Planned**.

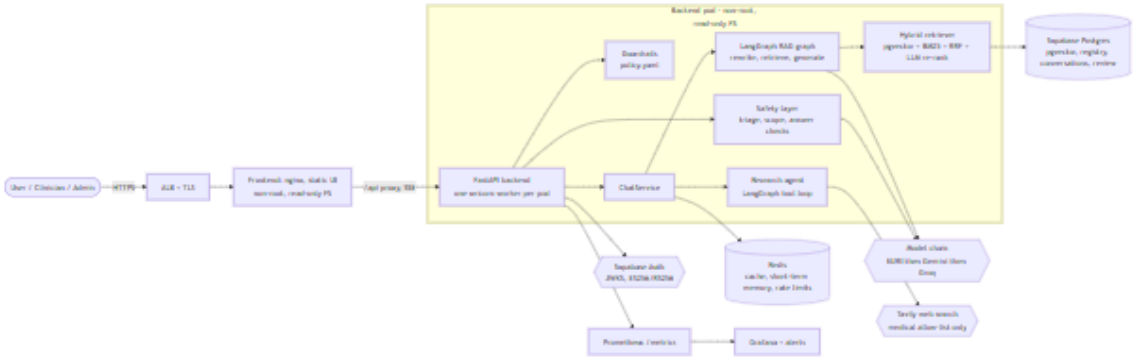
---

1. Purpose and scope

This HLD describes **what the system is made of and how the parts interact** to satisfy the PRD: system context, major components, end-to-end flows, data and integration architecture, security zones, deployment topology and the design of non-functional qualities. Component internals are in the LLD.

In scope: web frontend, API backend, agent pipeline, retrieval and ingestion, data stores, external integrations, observability, local and AWS deployment. **Out of scope:** clinical content authoring, EHR integration, mobile native apps.

2. System context



2.1 Actors

Actor	Interaction
Public user	Asks questions, uploads own documents, downloads reports, manages own history
Clinician	As user + research mode + clinician review
Admin	As clinician + trusted-corpus management + evaluation dashboard
Ops engineer	Monitors, deploys, responds to alerts (no product UI access needed)
CI/CD (GitHub Actions)	Builds, tests, scans and deploys via OIDC

2.2 External systems

System	Purpose	Protocol	Trust
Supabase Auth	Identity, JWT issuance (ES256)	HTTPS, JWKS	Trusted for identity
Supabase Postgres + pgvector	Vectors, registry, conversations, review, eval runs	Postgres TLS (pooler 6543)	Trusted store
EURI (OpenAI-compatible)	Primary chat + embeddings	HTTPS	Processor; receives redacted text
Google Gemini, Groq	Fallback chat	HTTPS	Processors; receive redacted text
Tavily	Web search (research mode)	HTTPS	Untrusted content source
Redis	Cache, short-term memory, rate limits	RESP (private network)	Trusted, ephemeral

2.3 System boundary

Everything inside the boundary is ours: the static frontend, the FastAPI backend and its agents, the policy and rules files, and the deployment manifests. Everything outside is reached only from `backend/app/rag/providers.py` (and the browser's direct call to Supabase Auth for sign-in).

---

3. Architecture overview and principles

#	Principle	Consequence in the design
P1	**Safety is structural**	Safety steps are mandatory code in the single answer path, not prompt text; request parameters cannot disable them
P2	**Evidence or nothing**	No passages -> no model call; every sentence verified against passages
P3	**Deterministic first, LLM second**	Rules (triage, scope, guardrails, checks) run deterministically; LLMs give second opinions at temperature 0
P4	**Least data**	PHI redacted locally before any egress, cache key, memory or log
P5	**Ports and adapters**	Use cases depend on interfaces; providers are adapters; every adapter has an offline fake
P6	**Fail safe**	Classifiers fail open to rules; the verifier fails closed; emergency notice survives outages
P7	**Secure by default**	Non-root, read-only containers; default-deny network; strict CSP; RBAC on every route
P8	**Observable**	Every step emits metrics and audit events without PHI

4. Logical components (container view)

Container	Technology	Responsibilities	Scales
Frontend	nginx-unprivileged + static HTML/CSS/JS	Serve UI with CSP; proxy `/api` (SSE unbuffered); hide backend internals	Horizontally (2-4)
Backend API	FastAPI (sync), uvicorn 1 worker/pod	Auth/RBAC, routes, error mapping, SSE, metrics, audit	Horizontally (2-8)
Agent pipeline (in backend)	LangGraph + LangChain	Triage, scope, rewrite, retrieve/re-rank, answer, verify, research	With backend
Ingestion (in backend)	pypdf, python-docx/pptx, openpyxl, RapidOCR	Load, OCR, guard, chunk, hash, embed, upsert	With backend (admin-triggered)
Vector + relational store	Supabase Postgres + pgvector	Chunks, FTS, registry, conversations, review, eval runs	Managed
Cache store	Redis 7 (no persistence)	Answer/LLM cache, short-term memory, rate limits	Single instance (HA later)
Model providers	EURI / Gemini / Groq	Chat + embeddings	External
Observability	Prometheus, Grafana, logs	Metrics, dashboards, alerts	External to app

Internal module map (backend): `api` (HTTP) -> `rag/service` (use case) -> `agents`, `safety`, `guardrails`, `rag/*` (domain) -> `rag/providers` (adapters) -> external systems. Details: Architecture.

5. End-to-end flows

5.1 Ask a question (happy path)

```
Browser        nginx        Backend API / ChatService                    Postgres   Models
  |--POST /api/chat/stream-->|----------------->|                          |          |
  |                          |                  |-guard input (local)      |          |
  |<---- phase: safety ------|------------------|-triage rules + LLM ------------------>|
  |                          |                  |-scope rules + LLM ------------------>|
  |<---- phase: searching ---|------------------|-embed query ------------------------->|
  |                          |                  |-vector + BM25 search --->|          |
  |                          |                  |-re-rank ------------------------------>|
  |<---- phase: generating --|------------------|-grounded answer ---------------------->|
  |<---- phase: verifying ---|------------------|-output guard, checks, (judge) -------->|
  |<---- answer (JSON) ------|------------------|-memory (encrypted) ----->|          |
```

5.2 Emergency

Triage fires on rules or LLM -> the emergency notice is created in code and placed first. Self-harm -> crisis resources only (no generation). If generation fails, the notice is still returned.

5.3 Upload and analyse a document

User `POST /api/documents` -> size/type checks -> loader (OCR for scans) -> document guard (neutralise injected lines, redact PHI) -> chunk -> hash -> embed changed chunks -> upsert into the **user** collection with `owner_id` -> later questions search trusted + own uploads only.

5.4 Research (clinician)

`POST /api/chat/stream?mode=research` -> same safety steps -> ReAct agent (verified tool model) chooses `search_corpus` / `search_web` (allow-listed) -> passages -> same grounded generation and verification.

5.5 Evidence report

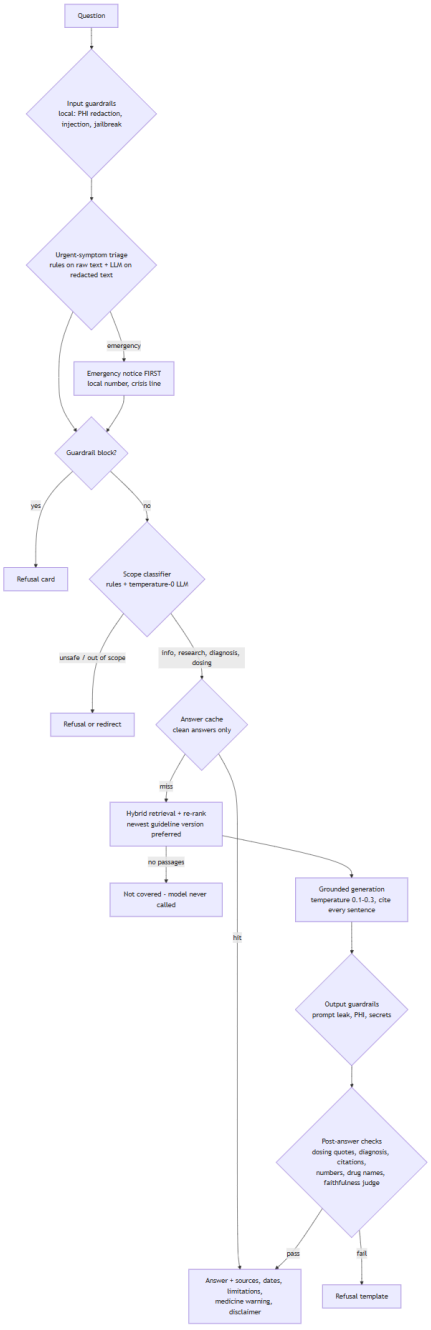
`GET /api/conversations/{id}/report` -> owner check -> stored redacted messages + citation provenance -> Markdown file download.

5.6 Corpus administration

Admin `POST /api/admin/documents` with provenance -> allow-list check -> ingestion into the **trusted** collection -> corpus version bump invalidates the answer cache.

5.7 Clinician review

A sample of clean answers is queued (redacted) -> clinicians record verdicts -> admins export.


Question
Input guardrails
local: PHI redaction,
injection, jailbreak
Urgent-symptom triage
rules on raw text + LLM on
redacted text
emergency
Emergency notice FIRST
local number, crisis line
Guardrail block?
yes
no
Refusal card
Scope classifier
rules + temperature-0 LLM
unsafe / out of scope
info, research, diagnosis,
dosing
Refusal or redirect
Answer cache
clean answers only
miss
Hybrid retrieval + re-rank
newest guideline version
preferred
no passages
Not covered - model never
called
hit
Grounded generation
temperature 0.1-0.3, cite
every sentence
Output guardrails
prompt leak, PHI, secrets
Post-answer checks
dosing quotes, diagnosis,
citations,
numbers, drug names,
faithfulness judge
pass
fail
Answer + sources, dates,
limitations,
medicine warning,
disclaimer
Refusal template

6. Data architecture

Store	Data	Classification	Protection	Retention
Postgres `langchain_pg_embedding`	Trusted chunks + vectors	Public/licensed content	DB encryption at rest, TLS	Until deleted by admin
Postgres `langchain_pg_embedding` (user collection)	User upload chunks (redacted)	Personal (minimised)	Owner-scoped queries, TLS	Until deleted by user
Postgres `medinfo_conversations/messages`	Redacted Q&A + citation provenance	Personal (minimised)	**Fernet app encryption** + DB encryption	30 days (configurable)
Postgres `medinfo_review_items`	Sampled redacted answers + verdicts	Personal (minimised)	Role-restricted	Per governance policy
Postgres `medinfo_ingest_registry`, `parent_chunks`, `eval_runs`	Metadata, parents, metrics	Internal	Role-restricted	Lifetime of corpus
Redis	Caches, recent turns, counters	Ephemeral	Private network, password, no persistence	TTL (minutes-hours)
Logs / audit	Events without question/answer text	Operational	Central log store	Per policy

Data residency: all stores in **ap-southeast-2** (Supabase project region). Details: Data model.

7. Integration architecture

Integration	Direction	Auth	Timeouts / retries	Failure behaviour
Supabase Auth (browser)	Browser -> Supabase	Publishable key	Browser default	Sign-in error shown
JWKS	Backend -> Supabase	None (public keys)	5 s, cached 600 s	401 if keys unavailable
Model chain (via LLM gateway)	Backend -> EURI/Gemini/Groq	API keys (secrets)	45 s, retries on unavailable, breakers; per-task token limits	Fallback; 429/502/503 mapping; over budget = failed call
Embeddings	Backend -> EURI	API key	4 retries	503; ingest fails without partial registry
Tavily	Backend -> Tavily	API key	15 s	Research falls back to corpus
Postgres	Backend -> Supabase pooler	Password in `DATABASE_URL`	10 s connect, pre-ping	Readiness 503
Redis	Backend -> Redis	Password	2 s	Caches/limits degrade gracefully

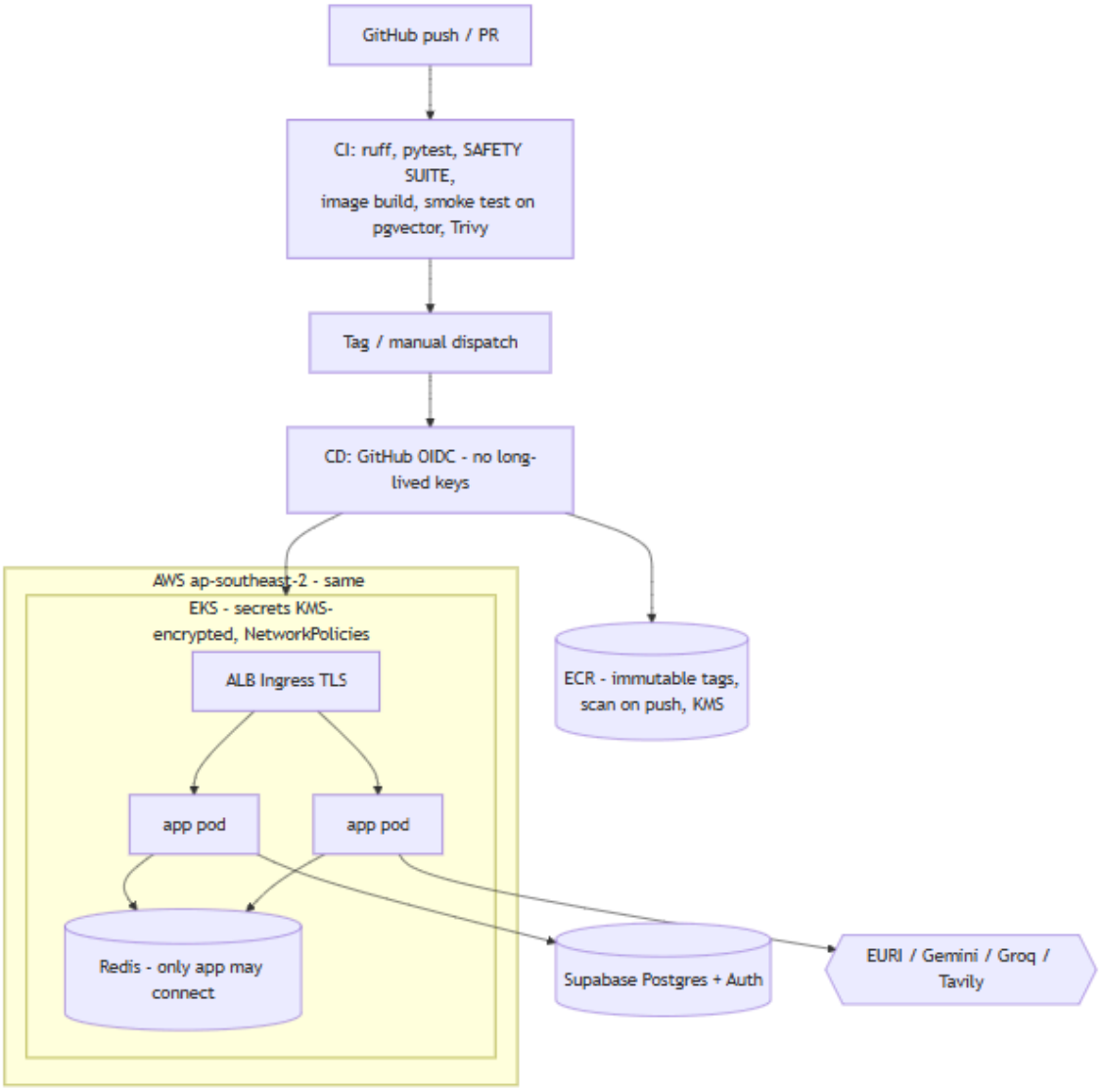
8. Security architecture

Zone	Contains	Controls
Internet	Browsers	TLS at ALB; CSP; no inline script
Edge	ALB, frontend pods	Only `/`, `/static`, `/api` exposed; `/metrics`, `/docs`, `/readyz` blocked
Application	Backend pods	JWT/JWKS, RBAC, rate limits, guardrails, audit; reachable only from frontend + monitoring
Data	Redis (cluster), Postgres (managed)	Redis reachable only from backend; Postgres TLS + password; app-level encryption
External processors	Model/search providers	Only redacted text leaves; DPAs required before real PHI

Trust boundaries crossed by untrusted content: user uploads, web results, model outputs - each is guarded before use. Details: Security & privacy.

9. Deployment architecture

Environment	Topology	Status
Local dev	uvicorn on 127.0.0.1:8030 serving API + UI; Supabase cloud DB; in-process cache	**Verified live**
Local compose	frontend (127.0.0.1:8080) -> backend -> Redis; optional pgvector + monitoring profiles	Written; build not yet run
AWS	ALB -> frontend pods -> backend pods -> Redis in EKS; Supabase DB; ECR; KMS	Planned (Terraform validated)



10. Non-functional design

Quality	Design	Target
Latency	Sequential stages today (running triage and scope classifiers in parallel is planned), answer cache, LLM prompt cache, re-ranker only over 12 candidates, streamed phases	p95 <= 15 s (target)
Throughput	Stateless pods; one worker each; HPA on CPU	Scale-out to 8 backend pods
Availability	>= 2 replicas, PDB, readiness gates, fallback models	99.5% monthly (target)
Resilience	Circuit breakers, retries, graceful degradation, emergency-first under outage	No unsafe failure mode
Capacity (estimate)	~3-5 chat calls + 1 embedding per question; ingestion 1 embedding call per 64 chunks	Quota planning per provider
Disaster recovery	Postgres PITR (to enable); IaC re-creates compute; Redis is disposable	RPO <= 24 h, RTO <= 4 h (targets)
Security	See section 8	Trivy clean (fixable HIGH/CRITICAL)
Accessibility	Semantic HTML, keyboard, contrast, reduced motion	WCAG 2.1 AA (target)

11. Observability design

- **Metrics** (Prometheus): HTTP latency per route template (p50/p95/p99), model and retrieval latency, triage events, scope decisions, refusals by reason, guardrail events, hallucination flags, cache hits, circuit state, fallbacks, rate limits, classifier errors.
- **Dashboards** (Grafana, as code) and **alerts** (triage spike, refusal shift, safety-check failures, open circuits, all models down, high p95, 5xx rate).
- **Logs**: JSON in containers, request id on every line, `extra=` fields; no PHI.
- **Audit**: one line per request and per action.

---

12. Technology stack and rationale

Layer	Choice	Why
Language	Python 3.11	Ecosystem for LLM/RAG; typed with pydantic
API	FastAPI (sync handlers)	Typed, fast, OpenAPI; sync keeps provider SDK usage simple
Orchestration	LangGraph	Explicit graphs for RAG and the ReAct loop; streaming of node updates
Models	OpenAI-compatible providers	One adapter, multiple vendors, fallback (ADR-001)
Vector DB	Postgres + pgvector (Supabase)	One store for vectors, FTS, metadata and app tables
Cache	Redis	TTL caches, counters, lists
UI	Plain JS + nginx	No build step, minimal attack surface, strict CSP
IaC / runtime	Terraform, EKS, Kubernetes	Standard, auditable, policy-enforceable
Quality	pytest, ruff, Playwright, ESLint, MLflow	Offline-first testing and tracked evaluation

13. Constraints, assumptions and risks

- **Constraints:** synthetic data only; provider free-tier quotas; Docker Desktop unavailable on the development machine (container builds deferred to CI).
- **Assumptions:** Supabase signs JWTs with ES256 (verified); OpenAI-compatible APIs remain stable.
- **Key risks:** see PRD section 17; architecture-specific: single Redis instance (cache loss is tolerated), single database region.

14. Design decisions

ADR	Decision
ADR-001	OpenAI-compatible provider chain; single embedding provider per collection
ADR-002	ReAct research agent gathers evidence only
ADR-003	Structured outputs; no exposed chain-of-thought
ADR-004	Allow-listed, clinician-only web access
ADR-005	Fail-open classifiers, fail-closed verifier, circuit breakers
ADR-006	In-process LLM gateway: task routing (fast/quality), per-task token limits, token metrics, query-embedding cache

15. Traceability (PRD goals -> HLD components)

PRD goal	Components
G1 Grounded answers	Retriever, answer agent, verifier
G2 Emergency first	Triage agent, templates, frontend emergency pin
G3 No diagnosis/prescribing	Scope agent, verifier, templates
G4 PHI protection	Guardrails, memory encryption, audit design, network zones
G5 Evidence reports	Report module, conversations store
G6 Measurable safety	Safety suite, evaluation runner, review queue, metrics/alerts
G7 Production operability	Containers, Kubernetes, Terraform, CI/CD, observability